

Compact Routing Guide

Scientific Data Analysis v2.1

A compact skill backend with plugin-eval evaluation

Maintenance: compact router, public snapshot in references, regression wrappers, and an initial benchmark

English publication edition: September 23, 2026

Translation of the preserved private v2.1 guide at commit 83dd0a5. Original TeX SHA-256: 62d605394261d9603a680bb93f62d5ecf13a02faa10609bd4d79979f322ab720. Private source: `skills/scientific-data-maintainer/.cache/active_references/guides/guia_scientific_data_analysis_v2_1.tex`.

Recorded results and closure steps are historical, not current verification. This release guide does not synchronize or modify Scientific Workbench; the app remains outside its scope. Historical commands are retained for traceability: use fresh output directories when reproducing them, without overwriting earlier evidence. The original remains unchanged privately.

Abstract

scientific-data-analysis v2.1 focuses on compact routing and evaluation after integrated v2.0. The skill remains the deterministic engine for scripts, JSON contracts, run bundles, artifacts, warnings, errors, provenance, and original preservation. **SKILL.md** becomes a compact router again; detailed operational instructions live in **references/**, and the generated registry snapshot is no longer always loaded.

Scientific execution remains local-first, with read-only originals or copied inputs. Scientific Workbench remains the v2.0-compatible app; v2.1 neither synchronizes nor modifies it.

Contents

1	Executive overview	1
2	From v2.0 to v2.1	1
3	What remains unchanged	1
4	The skill	1
5	Scientific Workbench	2
6	Integration responsibilities	2
7	v2.0 JSON contract	2
8	Artifact types	2
9	Run bundles	3
10	Local-first, optional cloud, and Codex bridge	3
10.1	Local-first	3
10.2	Optional cloud	3
10.3	Codex bridge	3
11	Safety	3
12	Workflows recorded in Phase 8	4
13	Limits	4
14	Historical verification commands	4
15	Troubleshooting	5
16	Plugin-eval	5
17	v2.1 checks	5
18	Synchronization	5
19	Historical decision	5

1. Executive overview

v2.1 consolidates the skill after integrated v2.0:

- Reduces `SKILL.md` to a compact router.
- Moves long routing, the public snapshot, and operational detail into `references/`.
- Preserves v1.8/v1.9/v2.0 app-readiness contracts.
- Adds `tests/` wrappers around `audit_*` regressions.
- Prepares an initial `plugin-eval` benchmark.
- Synchronizes the installed skill only after validating the editable tree.

This is a skill maintenance, activation, and evaluation release, with no new app feature phase or Scientific Workbench edits.

2. From v2.0 to v2.1

Area	v2.1 change
<code>SKILL.md</code>	Compact router for activation, safety, visible blocks, and key references.
Public snapshot	Moves to <code>references/public-surface-snapshot.md</code> .
Long routing	Preserved in <code>references/v2-1-expanded-skill-routing.md</code> .
Public contract	Registry, capabilities, and scripts remain compatible.
Regressions	Adds <code>tests/test_audit_regression_wrappers.py</code> for generic evaluators.
Plugin-eval	Before/after measurement targets lower active cost and an initial benchmark.
App	Scientific Workbench is not edited or synchronized; no new app version is declared.

3. What remains unchanged

- No capability renaming or scientific script contract changes.
- Optional backends do not become core dependencies.
- Domain-specific and legacy routes are not presented as general tools.
- Read-only/copy-based original protection remains mandatory.

4. The skill

The deterministic backend:

- Registers public capabilities.
- Inspects and processes data, documents, FITS, notebooks, tables, and packages.
- Emits `summary.json`, `manifest.json`, stdout, stderr, and typed artifacts.
- Declares `PASS`, `WARNING`, `BLOCKED_CONTROLADO`, `FAIL`, or audit status `ROTO`.
- Preserves originals through copies or separate outputs.
- Blocks cleanly for absent optional backends or unsafe inputs.

5. Scientific Workbench

The product interface:

- Loads the installed skill registry.
- Accepts file or folder attachments.
- Uses the local planner, skill router, or AI to propose workflows.
- Executes capabilities in traceable runs.
- Shows jobs, progress, cancellation, retry, resume, and support bundles.
- Discovers PNG, PDF, CSV/TSV, Markdown, notebook, JSON, and log previews.
- Provides guided controls for column selection, optional panels, and visual confirmation.

6. Integration responsibilities

Layer	Responsibility
Skill	Reproducible scripts, envelopes, run bundles, artifacts, and provenance.
Scientific Workbench	UI, planning, command review, processes, artifact indexing, and job recovery.
Local/cloud AI	Explanation, planning, and assistance; not the sole scientific source of truth.
Codex bridge	Extended reasoning, code, reporting, and filesystem tasks under explicit control.

7. v2.0 JSON contract

The app accepts legacy, v1.8/v1.9, and v2.0 payloads. Main v2.0 fields:

Field	Use
<code>contract_version</code>	2.0 for new envelopes; earlier compatibility remains.
<code>status</code> / <code>app_status</code>	Normalized job/UI state.
<code>command</code>	Redacted command with argv, cwd, and provenance.
<code>inputs</code> / <code>outputs</code>	Declared inputs and outputs.
<code>typed_artifacts</code>	Primary artifact list for Results.
<code>warnings</code> / <code>errors</code>	Human-readable and parseable messages.
<code>next_actions</code>	Useful continuation actions.
<code>original_modified</code>	<code>false</code> , <code>true</code> , or explained <code>unknown</code> .
<code>app_hints</code>	Short summary, severity, suggested previews, and tags.
<code>job_metadata</code>	Retry, cancellation, exposure, and estimates.
<code>safety</code>	Read-only/copy policy, secret redaction, and confirmations.

8. Artifact types

Scientific Workbench recognizes frozen v1.8/v1.9 types and v2.0 additions:

```
summary_json, manifest_json, report_md, preview_png, table_csv,
notebook_ipynb, log_txt, qa_report, handoff_bundle, unknown,
fits_product, fits_visual, metadata_json, preview_pdf,
edited_document, app_preview
```

Use **unknown** when an artifact cannot be typed accurately. Do not invent an ambiguous type to make the output appear more complete.

9. Run bundles

The preferred format remains:

```
run/  
  manifest.json  
  summary.json  
  stdout.txt  
  stderr.txt  
  command.txt  
  artifacts/  
  previews/  
  reports/  
  tables/  
  logs/  
  next_steps.md
```

The app can discover recursively but should prefer `typed_artifacts` and `manifest.json` when present.

10. Local-first, optional cloud, and Codex bridge

10.1. Local-first

The original guide recommends local Ollama: no paid API key is required, prompts/context remain on the local machine, and it supports planning, failure explanation, and result summaries.

10.2. Optional cloud

The guide lists OpenAI, Grok/xAI, and Gemini as optional providers for reasoning or writing. They can incur costs, require user keys, and do not execute capabilities themselves. Provider errors must be classified without leaking secrets. These are historical provider descriptions, not a current availability or pricing verification.

10.3. Codex bridge

Codex supports code, reports, refactors, and workspace reasoning. It does not replace the deterministic backend. Workflows must retain verifiable commands, outputs, artifacts, and logs.

11. Safety

The v2.0 integration requires:

- Read-only original inputs, with edits confined to copies or separate outputs.
- No API keys or secrets in logs, summaries, support bundles, or exportable prompts.
- Job records containing commands, statuses, warnings, errors, and next actions.
- Preflight and confirmation for optional/GUI backends, with clean blocking when prerequisites are absent.
- A release block for timeout, raw traceback, secret leakage, false success, or original modification.

12. Workflows recorded in Phase 8

Family	Status	Interpretation
Professional table	PASS	Parseable run bundle with artifacts.
Administrative document	WARNING	Useful intake with explicit limitations.
Inherited notebook	Controlled block	Stops interactive input or side effects before execution.
Corrupt container	WARNING	Recovers available content without false success.
Time series	WARNING	Visible date ambiguity.
Sensor/measurement	WARNING	Visible suspicious ranges.
FITS/RGB	WARNING	Successful RGB case; incomplete WCS warns.
Absent optional backend	Controlled block	Clean block with next actions.
DOCX roundtrip	PASS	Edits a confirmed copy, not the original.
Keynote/GUI	PASS	Preflight and confirmed export of a copy.

Controlled block denotes the original machine status `BLOCKED_CONTROLADO`.

13. Limits

- STILTS/TOPCAT, APT, Keynote, TEAREDUCE, IRAF, and iSTARMOD remain optional, expert, or legacy backends.
- Li I 6707.8 and SB2 tools are specialized, not general-purpose.
- GUI workflows need explicit confirmation and copied inputs.
- Cloud AI does not guarantee scientific accuracy.
- v2.0 is installed only after Phase 11 synchronizes the skill and validates the installed skill with Scientific Workbench.

14. Historical verification commands

The original workspace-specific paths are shown through aliases: `$PRIVATE_WORKSPACE` is the original editable workspace; `$DATANALYSIS_PYTHON` is its explicitly selected scientific interpreter. The external dual-gate script belongs to that workspace and is not supplied by this source publication. These are historical reproduction steps, not the current public installation procedure.

Main dual gate:

```
cd "$PRIVATE_WORKSPACE"
./run_v2_0_dual_gate.sh
```

Skill check:

```
cd "$PRIVATE_WORKSPACE/skill_work/scientific-data-analysis"
"$DATANALYSIS_PYTHON" scripts/sync_public_surface_docs.py --check
```

App check:

```
cd "$PRIVATE_WORKSPACE/ScientificWorkbench"
SKIP_UI_SMOKE=1 RUN_DOCUS_BENCHMARK=0 ./script/run_quality_gate.sh
```

15. Troubleshooting

Symptom	Next step
Absent optional backend	Accept <code>BLOCKED_CONTROLADO</code> ; inspect <code>preflight</code> and <code>next_actions</code> .
Output conflict	Use a fresh temporary folder or change <code>-outdir</code> .
Notebook with <code>input()</code>	Execute a copy with declared values or block before execution.
Missing preview	Inspect <code>manifest.json</code> , <code>typed_artifacts</code> , and <code>previews/</code> .
Cloud provider failure	Fall back to Ollama/local or the skill router; do not leak keys in logs.
Ambiguous result	Retain <code>WARNING</code> and explain the limit instead of silently reporting success.

16. Plugin-eval

The prior installed-skill evaluation identified excessive active context: `SKILL.md` was too large, and the evaluator did not recognize `audit_*` scripts as conventional tests.

v2.1 responds with a compact router, a generated snapshot outside `SKILL.md`, an expanded reference preserving operational detail, `unittest` wrappers under `tests/`, an initial `plugin-eval init-benchmark`, and final `plugin-eval analyze`.

17. v2.1 checks

```
cd "$PRIVATE_WORKSPACE/skill_work/scientific-data-analysis"
"$DATANALYSIS_PYTHON" scripts/sync_public_surface_docs.py --check
"$DATANALYSIS_PYTHON" scripts/skill_surface_audit.py --skip-hygiene
"$DATANALYSIS_PYTHON" -m unittest tests.test_audit_regression_wrappers -v
```

18. Synchronization

Final v2.1 synchronization copies the editable skill into the installed skill only after green gates. It excludes `tmp/`, `__pycache__`, `*.pyc`, and `.DS_Store`.

Scientific Workbench is not synchronized in v2.1. App consumption of these changes requires a later explicitly scoped phase.

19. Historical decision

v2.1 is ready only when the editable tree passes checks, the PDF compiles, the installed skill is synchronized from the validated editable tree, post-sync checks pass, `plugin-eval init-benchmark` creates its initial configuration, and final `plugin-eval` evaluation confirms lower active cost.